

15차시	총10문제		연습: ☐	과제 : ☒	평가 : ☐
1. 파이썬에서 이터레이터(iterator)의 주요 특징은 무엇인가요? a) 반복 가능한 객체를 생성한다. b) next() 메서드로 값을 순차적으로 반환한다. c) 리스트나 튜플만 이터레이터가 될 수 있다. d) 이터레이터는 재사용 가능하다. 답 b 2. 다음 코드의 실행 결과는 무엇인가요? <pre>nums = [1, 2, 3] it = iter(nums) print(next(it)) print(next(it))</pre> a) 1 b) 1, 2 c) 2 d) 에러 발생 답 b					

3. 다음 코드의 출력 결과는 무엇인가요?

```
def my_gen():  
    yield 1  
    yield 2  
    yield 3  
gen = my_gen()  
print(next(gen))  
print(next(gen))
```

- a) 1
- b) 1, 2
- c) 1, 3
- d) 에러 발생

답 b

4. 다음 중 제너레이터 표현식의 올바른 예는 무엇인가요?

- a) `[x * 2 for x in range(5)]`
- b) `(x * 2 for x in range(5))`
- c) `{x * 2 for x in range(5)}`
- d) `lambda x: x * 2 for x in range(5)`

답: b

5. 다음 코드의 실행 결과는 무엇인가요?

```
def countdown(n):  
    while n > 0:  
        yield n  
        n -= 1  
  
gen = countdown(3)  
for x in gen:  
    print(x, end=" ")
```

- a) 3 2 1
- b) 1 2 3
- c) 에러 발생
- d) 3 3 3

답: a

6. 주어진 리스트를 이터레이터로 변환하고, 각 요소를 하나씩 출력하는 프로그램을 작성하세요.

```
numbers = [1, 2, 3, 4, 5]
```

답:

```
numbers = [1, 2, 3, 4, 5]
```

```
ia = iter(numbers)
```

```
print(next(ia))
```

```
for i in ia:
```

```
    print(i)
```

7. 주어진 리스트에서 next() 함수를 사용하여 각 요소를 하나씩 출력하세요. StopIteration 예외를 처리하여 출력이 끝날 때까지 반복되도록 하세요.

```
fruits = ["apple", "banana", "cherry"]
```

답:

```
fruits = ["apple", "banana", "cherry"]
```

```
it = iter(fruits)
```

```
for i in range(len(fruits)):
```

```
    try:
```

```
        print(next(it))
```

```
    except StopIteration:
```

```
        break
```

8. 0부터 9까지의 숫자를 이터레이터로 순회하며, 각 숫자의 제곱을 출력하는 프로그램을 작성하세요.

답:

```
ia = (i*i for i in range(10))
```

```
print(next(ia))
```

```
for i in ia:
```

```
    print(i)
```

```
print(next(ia))
```

9, 0부터 10까지의 숫자 중 짝수만 출력하는 프로그램을 작성하세요. 이때, 이터레이터를 사용해야 합니다.

답:

```
ia = (i for i in range(10) if i % 2 == 0)
print(next(ia))
for i in ia:
    print(i)
print(next(ia))
```

10. MyRange라는 이름의 클래스를 만들어, range()와 같은 기능을 하는 이터레이터를 작성하세요. start, stop, step을 인자로 받아서 동작해야 하며, __iter__()와 __next__() 메서드를 구현해야 합니다.

답:

```
class MyRange():
    def __init__(self, start, stop, step=1):

        self.start = start
        self.stop = stop
        self.step = step
        self.current = start
    def __iter__(self):
        return self
    def __next__(self):
        if (self.step > 0 and self.current >= self.stop) or \
            (self.step < 0 and self.current <= self.stop):
            raise StopIteration
        result = self.current
        self.current += self.step
        return result
```

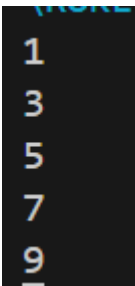
```
ra = MyRange(1,10,2)
```

```
print(next(ra))
```

```
for i in ra:
```

```
    print(i)
```

```
print(next(ra))
```



```
MORE
1
3
5
7
9
_
```

